



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2513 - TECNOLOGÍAS Y APLICACIONES WEB

Entrega 5

Tecnologías y Aplicaciones Web

2º semestre 2026 - Profesor Hernán Cabrera

Fecha de Entrega: Domingo 22 de Noviembre a las 23:59 hrs.

Introducción

Hemos llegado a la etapa final del curso. Tras diseñar la arquitectura (E_1), maquetar la interfaz en React (E_2), estructurar el backend con persistencia relacional (E_3) y programar la lógica del juego con autenticación y WebSockets (E_4), es momento de consolidar todo en un **producto web completo, pulido y en producción**.

El objetivo principal de esta **Entrega 5** es lograr la **integración total y fluida entre el Frontend y el Backend**, garantizando que la aplicación opere de extremo a extremo en la nube (PaaS). Además, deberán implementar los paneles de gestión y auditoría, asegurar los más altos estándares de calidad de código mediante el cumplimiento estricto de la guía de estilos de **Airbnb (ESLint)**, incorporar al menos una **funcionalidad avanzada/extra** y entregar la documentación final exhaustiva de su aplicación.

Indicaciones Generales

Para esta entrega final, su proyecto debe comportarse como una aplicación web profesional accesible públicamente por cualquier usuario de Internet, cumpliendo con los siguientes requerimientos:

- Integración Front-End y Back-End Total:** Comunicación continua vía HTTP REST y WebSockets, sincronización visual inmediata del estado de las partidas y manejo de sesiones con JWT en el cliente.
- Despliegue Completo en Producción:** Frontend desplegado en PaaS (Netlify o Vercel) consumiendo en vivo la API pública y base de datos desplegadas en la nube (Render, Supabase o similar).
- Panel de Administración Funcional:** Vista protegida que permita a usuarios con rol administrador auditar, gestionar y dar de baja usuarios, partidas o acciones del sistema.
- Panel de Resumen de Acciones por Usuario:** Bitácora visual donde cada jugador pueda consultar su historial de transacciones, jugadas y desempeño en partidas previas.

- e. **Calidad de Código y Linter (Airbnb):** Cumplimiento estricto del estándar de ESLint en Frontend y Backend sin errores ni advertencias de sintaxis.
- f. **Funcionalidad Extra / Avanzada:** Implementación de una característica no trivial que agregue valor diferenciador a su juego.
- g. **Documentación Final y Cierre de Proyecto:** README exhaustivo, diagrama ER final actualizado, especificación de eventos WebSocket y Tablero Kanban concluido.

Requerimientos Detallados

1. Integración Total Front-Back y Sincronización en Tiempo Real

- **Flujo Completo de Partida:** Dos o más usuarios deben poder registrarse, iniciar sesión, unirse a una sala, disputar una partida completa por turnos (respetando las reglas) y alcanzar una condición de término (victoria, derrota o empate) sin errores de sincronización.
- **Manejo de Sesión y Rutas Protegidas en React:** El token JWT recibido en el login debe almacenarse adecuadamente (e.g., `localStorage` o contexto global), adjuntándose en cada petición HTTP y socket autenticado. Las rutas restringidas en el cliente deben redirigir a los usuarios no autenticados.
- **Experiencia en Tiempo Real (WebSockets):** El tablero y los paneles informativos deben actualizarse automáticamente ante las jugadas de los demás participantes sin requerir que el usuario recargue manualmente el navegador.

2. Despliegue Completo en la Nube (PaaS)

- **Frontend:** Desplegado y funcionando en una plataforma como **Netlify** o **Vercel**.
- **Backend:** Desplegado en **Render** (o servicio equivalente) conectado a una base de datos **PostgreSQL** en la nube (como **Supabase** o **Render**).
- **Operación en Producción:** El frontend desplegado debe comunicarse exclusivamente con la URL pública de la API en producción (no con `localhost`). Los enlaces públicos deben figurar de forma destacada en la cabecera del `README.md`.

3. Panel de Administración Funcional

Se debe implementar una vista exclusiva y protegida para usuarios con rol de **Administrador**:

- Visualización de la lista de usuarios registrados y partidas en curso/finalizadas.
- Capacidad de auditar acciones y ejecutar operaciones administrativas (e.g., suspender o eliminar usuarios, cancelar partidas problemáticas o restablecer configuraciones).
- El acceso debe estar bloqueado y denegado tanto desde la interfaz de React como en los endpoints del servidor para usuarios no administradores.

4. Panel de Resumen de Acciones / Historial de Usuario

Cada usuario autenticado debe disponer de una sección personal que actúe como un registro o estado de cuenta:

- Historial cronológico de partidas jugadas, resultados obtenidos (ganadas/perdidas) y puntuaciones.
- Detalle o bitácora de transacciones relevantes (recursos obtenidos, casillas conquistadas, compras realizadas).

5. Calidad de Código y Guía de Estilo Airbnb (ESLint)

- Tanto el repositorio de Frontend como el de Backend deben contar con la configuración de **ESLint** siguiendo la guía de estilo de **Airbnb**.
- La ejecución de los scripts de linting (`npm run lint`) debe finalizar con **cero errores** en ambos entornos.
- Se verificará el orden, legibilidad, estructura modular (MVC), ausencia de código muerto o comentado y la total ausencia de estilos *inline* en React.

6. Funcionalidades Extras (Bonus y Salón de la Fama)

Los grupos que cuenten con todos los requerimientos mínimos podrán optar a implementar **UNA** de las siguientes funcionalidades avanzadas (o una propuesta equivalente acordada previamente con su ayudante):

1. **Multipartida Simultánea:** Capacidad de que varios jugadores participen e interactúen en múltiples partidas administradas en paralelo por el servidor.
2. **Reproducción Animada de Partidas:** Visualizador interactivo que permita reconstruir paso a paso en el tablero una partida ya finalizada desde su estado inicial.
3. **Dashboard Estadístico Global:** Panel con métricas avanzadas, tablas de clasificación (*leaderboards*), gráficos de rendimiento y atributos comparativos de la comunidad.
4. **Selección y Creación de Tableros:** Permitir que los usuarios seleccionen entre distintas configuraciones de mapas/tableros o creen escenarios personalizados antes de iniciar la partida.

Candidatura al Salón de la Fama: Los proyectos sobresalientes que destaquen por su diseño visual, estabilidad técnica, originalidad y despliegue impecable serán postulados por el cuerpo de ayudantes para optar a la distinción máxima.

7. Documentación Final y Gestión del Proyecto

- **README.md Final:** Documento exhaustivo que contenga enlaces a los despliegues, credenciales de prueba (para usuario normal y administrador), instrucciones de ejecución local, variables de entorno requeridas (`.env.example`) y resumen de funcionalidades implementadas.
- **Modelo ER Final Actualizado:** Diagrama entidad-relación en `/docs` que refleje fielmente el estado final de las tablas y asociaciones en PostgreSQL.

- **Documentación de WebSockets:** Explicación de los eventos y canales implementados para la sincronización en tiempo real.
- **Tablero Kanban Finalizado:** Tablero en GitHub Projects, Trello o Jira con todas las historias de usuario y tareas en estado “Finalizado”.

Condiciones y Restricciones

- Stack tecnológico oficial: **Node.js, Koa, PostgreSQL, Sequelize, React, React Router, WebSockets** y **ESLint**.
- Prohibido el uso de TypeScript o precompiladores ajenos a JavaScript estándar.
- Prohibido el uso de frameworks CSS (Bootstrap, Tailwind, etc.) o estilos *inline*.
- **Seguridad:** Ningún secreto, credencial de base de datos ni archivo **.env** debe figurar en los repositorios de GitHub.
- **Trazabilidad en GitHub:** Se auditará la participación individual mediante commits y Pull Requests. Integrantes con participación insuficiente serán penalizados según el reglamento general.
- **No se aceptarán entregas por correo electrónico ni fuera del plazo límite.**

Sobre la Calificación

La calificación final evaluará la completitud del producto, la robustez de la integración, la experiencia de usuario, la estabilidad del despliegue en la nube y la adherencia a buenas prácticas de ingeniería de software.

Criterios de Evaluación

Criterio	Esencial	Intermedio	Completo	Pje.
1. Integración Total Front-Back	Fallos graves de comunicación HTTP	Flujo básico funciona pero con desincronizaciones	Integración fluida HTTP y WebSockets, ciclo de partida completo y coherente	15
2. Despliegue en Producción (PaaS)	Despliegue caído o solo en local	Solo uno de los ambientes desplegado	Frontend (Netlify/Vercel) y Backend+BD (PaaS) accesibles públicamente y conectados	15
3. Panel de Administración	Vista inexistente o sin control de acceso	Panel visible pero con acciones incompletas	Vista protegida funcional exclusiva para Admin (gestión y auditoría de usuarios/partidas)	10
4. Resumen de Acciones / Historial	Información estática o incompleta	Muestra partidas pero sin detalle de acciones	Historial dinámico completo por usuario (partidas, transacciones, jugadas previas)	10
5. Autenticación y Rutas Protegidas	Sesión volátil o rutas vulnerables	Protección en backend pero no en cliente	Sistema integral de autenticación con JWT, bcrypt y rutas protegidas en React y Koa	10
6. Lógica de Juego y Servidor Árbitro	El cliente manipula reglas o turnos	Reglas básicas sin cubrir casos borde	Servidor como árbitro estricto, gestión de turnos, persistencia en PostgreSQL	10
7. Calidad de Código y ESLint (Airbnb)	Múltiples advertencias o sin linter	Linter configurado con errores menores	ESLint configurado y cumpliendo 100 % estándar Airbnb sin errores en front y back	10
8. Documentación Final y Diagrama ER	README incompleto o sin credenciales	Instrucciones básicas sin diagrama final	README exhaustivo con guías, credenciales, documentación de sockets y ER actualizado	8
9. Funcionalidad Extra / Avanzada	Propuesta incompleta o trivial	Funcionalidad operativa con errores visuales	Característica avanzada (multipartida, animaciones, dashboard, etc.) 100 % funcional	7
10. Tablero Kanban y Trazabilidad	Tablero incompleto o desactualizado	Tareas cerradas sin detalle de avance	Tablero finalizado con trazabilidad total y balance colaborativo en GitHub	5

Puntaje Total: 100 puntos

Requisitos Mínimos para Nota 4.0

Para optar a una calificación igual o superior a 4.0, el grupo debe cumplir obligatoriamente en nivel **Completo** con los siguientes ítems:

1. Integración total Front-End y Back-End con ciclo de juego completo.
2. Despliegue funcional del Frontend y Backend en plataformas PaaS en producción.
3. Panel de Administración funcional y protegido por roles.
4. Panel de Resumen de Acciones e Historial por usuario.
5. Autenticación segura con JWT y rutas protegidas en cliente y servidor.

Para optar al salón de la fama, cumplir obligatoriamente en nivel **Completo** con los siguientes ítems:

6. Servidor como árbitro de la lógica de negocio y persistencia en base de datos.
7. Calidad de Código y ESLint (Airbnb)
8. Documentación final completa en README y Modelo ER actualizado.

Política de Referencia y Uso de Inteligencia Artificial

En el desarrollo del proyecto grupal, los estudiantes podrán utilizar material proporcionado por el equipo docente, recursos disponibles en internet y herramientas basadas en Inteligencia Artificial (por ejemplo, ChatGPT, Claude, GitHub Copilot, entre otras). Sin embargo, todas las fuentes utilizadas deben ser debidamente referenciadas y el equipo debe demostrar un **aporte propio significativo** en la solución implementada.

Si se utiliza Inteligencia Artificial, se deberá indicar de forma explícita, transparente y detallada en una sección dedicada del archivo `README.md` de la entrega (o en un documento anexo dentro de la carpeta `/docs`), incluyendo:

- **Alcance y contexto:** Indicar con precisión qué partes del trabajo, componentes, estilos o funciones fueron apoyadas o generadas mediante IA.
- **Registro de interacción:** Adjuntar los *prompts* utilizados, los fragmentos (*snippets*) de código entregados por la herramienta y los enlaces públicos a las conversaciones o **capturas de pantalla legibles de los chats**.
- **Aporte y análisis crítico:** Explicar qué modificaciones, adaptaciones, correcciones e integraciones realizó el equipo sobre dicho contenido para incorporarlo al proyecto.

Régimen de Penalizaciones y Descuentos:

- En caso de que el aporte de los integrantes no sea considerado sustancial (por ejemplo, si la mayor parte del trabajo corresponde a código generado automáticamente sin comprensión ni personalización por parte del grupo), la nota podrá ser rebajada según el criterio del equipo docente.

- **Uso no referenciado o referenciado incorrectamente:** El uso de herramientas de IA sin su debida declaración, o con un reporte incompleto (e.g., no adjuntar capturas, prompts o no transparentar el alcance real de su uso), constituye una falta grave a esta política y se aplicará un **descuento directo de entre 0 a 100 puntos sobre la calificación final de la entrega**, dependiendo de la magnitud y gravedad de la omisión detectada. El uso no declarado de IA podrá derivar adicionalmente en una calificación final de nota 1,1, además de las medidas disciplinarias que determine el equipo docente.